TIKO — Deployment Plan

Oct 10, 2026 · @ki

1. Overview and principles

TIKO is deployed as one Next.js app on serverless hosting, backed by a managed Postgres database and a few outside services, and released in small, checked steps that can each be reversed. This plan says where it runs, how a change reaches users, how secrets and data are protected on the way, and what to do when something goes wrong. It follows the Architecture Document, Technical Specification, Database Design and Testing Plan; where the Tech Stack document (not available when this was written) differs, it wins.

Principles

- **Same path every time.** Every change, including a one-line fix, goes through the same pipeline; nothing is edited on a server by hand.
- **Small and reversible.** Releases are small; a bad build is undone by redeploying the previous one. Database changes are written so the old code still works.
- **Secrets stay out of the repository** and out of the browser bundle; one set per environment.
- **Real data is treated as sensitive in every place it exists,** including backups and the beta.
- **Devices are part of the system.** Because every device keeps its own copy of the user's data, a server outage or rollback does not stop logging, and a restore can be repaired from devices.

Release stages and gates (from the PRD)

Stage	Build weeks	Gate to start the next stage	Release criteria
MVP beta	1 to 8	10 beta users log 4 or more days a week	All P0 requirements pass; NFR-01, 02, 04, 05, 06, 08, 11 pass; no open severe defects; privacy and disclaimer text reviewed

Stage	Build weeks	Gate to start the next stage	Release criteria
MVP public	After the beta gate	Public launch decision	MVP gate met; chosen P1 requirements done; DPDP and GDPR review complete (NFR-16); TIKO name check done
V1	9 to 16	Day-30 retention near 20%	V1 gate met; all V1 requirements pass; payments tested end to end; export and deletion tested
V2	17 to 26	Retention and activation targets hold for two consecutive months	Native wrapper passes store review

Durations are estimates for a small team and stretch if one person builds part-time.

2. Environments and infrastructure

Four environments run the same code with different data and secrets. Every service named is a proposal from the Technical Specification and can be swapped without changing the app; the PRD lists a fallback for each.

Service	Proposed choice	Used for	If it fails (PRD fallbacks for hosting, sign-in, email, payments and name; others proposed)
Source control and CI	GitHub and GitHub Actions	Code, pull-request checks, deploy triggers	Any CI that runs the same scripts
Hosting	Vercel	Web app, API routes, preview deploys	Move to another provider; export is built in
Database	Neon or Supabase Postgres	All synced data, change feed	Move to another Postgres provider

Service	Proposed choice	Used for	If it fails (PRD fallbacks for hosting, sign-in, email, payments and name; others proposed)
Sign-in	Auth.js, or Supabase Auth with Supabase	Magic link and Google	Magic link still works if Google fails
Email	Resend	Magic links and fallback reminders	Switch email provider
Scheduler	External cron such as Upstash QStash or Supabase pg_cron	Reminders every 5 minutes, nightly purge, recurring expenses	Email and in-app reminders continue to work on app open
Rate-limit store	Redis-style store such as Upstash	Sign-in and write limits	Fall back to stricter database-backed limits
Object storage	Vercel Blob or an S3-compatible bucket	Treat photos	Photos wait on the device and upload later
Error monitoring	Sentry	Errors with personal content scrubbed	Logs only
Uptime monitoring	Any external checker	Availability target of 99.5% a month (to validate)	Second checker
Payments (V1)	Razorpay	Plus plan	Delay billing; the app stays free
Domain and DNS	A registrar and DNS host	The TIKO name; email authentication records	Rename; the logo and docs are easy to update

The four environments

Environment	Hosting	Database	Data	Who can reach it
Local	Developer machine	Local Postgres container	Invented fixtures	Developer

Environment	Hosting	Database	Data	Who can reach it
Preview	One per pull request	A throwaway or branched database	Invented fixtures	Developer; link not public
Staging	Its own project	Its own database	Invented data and test accounts only	Developer and invited reviewers
Production	Its own project	Its own database with backups	Real user data	Everyone (public launch) or invited testers (beta)

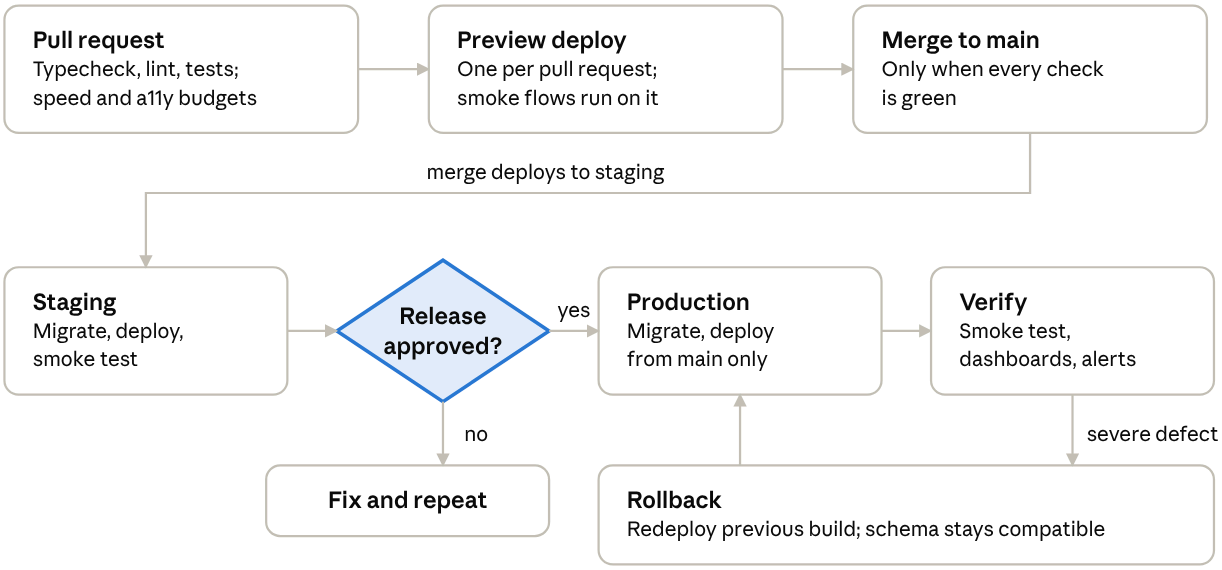
Separation rules

- Each environment has its own database, secrets, storage and scheduler; nothing is shared between staging and production.
- Real user data never leaves production. Lower environments use invented data, including invented journal and mood text.
- Staging uses a test Google sign-in and a test inbox, so no real person receives a staging email or push.
- Alerts and paging go only to production; staging failures show in the pipeline.
- Choose the region nearest the users (India) for hosting and database where the providers offer it. Where personal data may be stored is for the DPDP and GDPR review to confirm.

Beta runs on production. Beta testers enter real data, so the beta uses the production environment with invites only, not staging. This differs from the Architecture Document, which named staging for beta testers; keep the two aligned.

3. Release pipeline

Nothing reaches production without a green build, a staging check and an approval



release pipeline · pull request to production, one approval gate

A change passes the same checks in the same order. The only human step is the approval before production, and a severe defect found afterwards sends the release back through rollback.

Rules

- The main branch is protected: no direct pushes, pull requests need every required check green, and the check list is the pull-request stage of the Testing Plan.
- Production deploys only from main and only after a green staging smoke test. The approval is a manual step until the beta gate is met; after that it may become automatic for small fixes and stay manual for anything that changes the database or the engine.
- Each release is a tagged commit with a short note, so any production state can be named and redeployed.
- Order inside a deploy: apply database migrations first, then start the new code, then run the smoke test. Migrations are written so the previous code keeps working (add, backfill, then remove in a later release).
- A deploy that fails its smoke test is rolled back at once, before anyone is told it shipped.

- Nightly and pre-release checks (cross-browser, offline and sync scenarios, load, security scans) run on staging and feed the approval decision; they are not part of the quick path.

4. Configuration and secrets

Secrets live only in the hosting provider's environment settings, one set per environment, and never in the repository, the build log or the browser bundle. Only values safe to show a user carry the public prefix that exposes them to the browser. Names below are proposals and follow the chosen libraries.

Setting	Purpose	Rotation and handling
Session secret	Signs session cookies	Rotating signs everyone out; do it on suspicion of leak or yearly
App database URL	Connects as the app role (no schema rights); use the pooled connection string for serverless	Rotate yearly or on suspicion
Migration database URL	Connects as the migration role; available only to the deploy step	Never present at runtime
Master encryption key	Wraps each user's journal key	**Back up offline in two places.** Losing it makes every journal unreadable. Rotate by adding a new key version and re-wrapping user keys, not by replacing it
Email API key and from address	Magic links and fallback reminders	Rotate yearly; the from address must match the verified domain
Google client id and secret	Google sign-in	One client per environment; rotate if exposed

Setting	Purpose	Rotation and handling
Push public and private keys, contact address	Web push signing	**Do not rotate casually:** a new key invalidates every existing subscription and users must resubscribe
Cron secret	Authenticates scheduled-job calls	Rotate yearly; update the scheduler at the same time
Rate-limit store URL and token	Sign-in and write limits	Rotate yearly
Object storage credentials	Treat photos	Least privilege: write and read the one bucket
Error monitoring DSN	Sentry	Not secret but kept per environment so staging errors stay apart
Analytics key	Product events through the allowlist	Per environment
Razorpay key id, key secret and webhook secret (V1)	Plus plan billing	Test keys in staging; live keys only in production; rotate on suspicion

Rules

1. A secret is added to an environment through the provider's settings or a secrets tool, never committed, never pasted into a ticket or chat.
2. A secret scan runs on every pull request and on the built output; a finding blocks the merge.
3. Only the developer has access to production secrets; access is reviewed before each release.
4. A feature flag table or settings in the database may hold non-secret switches (for example turning recurring expenses or push on or off) so a risky feature can be disabled without a deploy. This is an option, not yet a requirement.
5. The list above is checked at every release: each environment must have every setting it needs, and no staging value equals a production value (a start-up check refuses to run if they match).

Key custody. The master encryption key is the one item whose loss cannot be repaired. Keep one copy in the hosting provider's secret store and one offline copy stored apart from the database account, and test recovery once before the beta by decrypting a staging journal entry using only the offline copy.

5. Database: migrations, roles, backups and restore

The database is the one part of the system that cannot be redeployed, so it gets the most care. The schema is in the Database Design; this section covers how it is changed and protected.

Migrations

- Forward-only SQL migrations (Drizzle Kit) live in the repository; an applied migration is never edited.
- They run in the deploy step, before the new code starts, using the migration role.
- **Expand, then contract.** First add nullable columns or new tables and deploy code that writes both shapes; backfill; only in a later release drop what is unused. Cached clients keep running old code after a deploy, so the old shape must keep working for at least one release.
- Each sync request carries a schema version. When a release raises the minimum, older clients get `client_too_old` and are asked to reload.
- CI applies every migration to an empty Postgres and then runs the constraint, sync-idempotency and account-deletion tests against it.
- A migration that rewrites a large table is run in batches, outside peak hours, and tested on a staging copy of similar size first.
- A check in CI fails if any table with a user id lacks row-level security and its policy.

Roles

Role	Rights	Used by
Migration	Owns the schema; can alter it	Deploy step only
App	Read and write on tables; no schema changes; row-level security applies	Running app
Reporting (optional)	Read-only; no access to journal, mood or expense tables	Analytics, if ever needed

Backups and restore

- Daily automatic backups, kept 30 days at most. A deleted account's data must be gone from live tables within 30 days and from backups when they age out, so retention may not exceed the 30-day promise (NFR-07). If the provider offers point-in-time recovery, keep its window within the same limit.
- Confirm with the provider that backups are encrypted at rest; the master encryption key is never stored in the database or its backups.
- **Restore drill:** before public launch, restore the latest backup into a separate database, run the integrity checks (row counts per table, a sample of journal decryptions) and time the procedure. Repeat after any change of provider or plan, and at least every quarter once live.
- **Restore with local-first devices.** After a restore the server may hold older data than some devices. Devices still have their copies, so they can repair it, but the sync API does not yet tell a device that the server was rolled back. Proposed: add a database epoch to sync responses; when it changes, a device re-sends its recent local data, which is safe because every mutation is idempotent. This is not in the API Documentation yet.

Connections and idle behavior. Serverless functions open many short connections, so use the provider's pooled connection string. Some free database plans pause or slow when idle, which adds delay to the first request and can miss reminder runs; check this for the chosen plan before the beta and use a paid plan for production if it applies.

Seed data. Only the entitlements rows (free and plus) are seeded in the database; templates, starter treats and core habits ship with the app. Categories are created for each user on first sync.

6. Launch setup

These items are set up once per environment, before the beta for staging and production, and verified again before public launch. Each has a check so "done" can be proven.

Item	What to set up	How to check it
Name, domain and DNS	Complete the TIKO trademark and domain check (a PRD dependency and a public-launch criterion); register the domain; point it at the host with managed TLS	The app loads over HTTPS only; plain HTTP redirects; HSTS and a strict Content Security Policy are present
Email delivery	Verify the sending domain with the provider; publish SPF, DKIM and DMARC records; use one from address	A magic link reaches major mailbox providers' inboxes (not spam) in under 60 seconds (story A3); a test email passes an authentication check
Google sign-in	Create the OAuth client; set the consent screen and the redirect addresses for each environment; staging uses a test account and production needs the consent screen published	Sign in on staging and production; an existing email links to one account
Web push	Generate the signing keys once; serve over HTTPS; register the service worker at the site root	A reminder arrives on an Android phone and on an iPhone with the app installed; a removed subscription falls back to email
PWA	Web manifest with name, icons, start page and scope; service worker with a version and an update flow	Install from Chrome and Safari; open offline after one visit; a new release prompts a reload without losing queued entries
Scheduler	Register the reminder, day-close, recurring, purge and (V1) recap jobs with the cron secret	Each job logs a heartbeat; a missed run raises an alert; running a job twice changes nothing
Object storage	Private bucket for treat photos; upload only through the API	A photo uploads and displays; a direct public listing fails
Error and event tracking	Sentry project per environment with scrubbing; analytics with the event allowlist	A test error shows no personal content; an event carrying free text or an amount is dropped

Item	What to set up	How to check it
Legal and help pages	Privacy policy, terms, the help and privacy page that explains what data TIKO keeps (FR-SET-05), a contact and grievance route, the self-care disclaimer	Reviewed before public launch (NFR-16); linked from onboarding and Settings
Support card content	Helpline list per country, confirmed and dated	Content reviewer signs off; shown correctly for each supported country
Support and contact	A monitored address for user reports and data requests	A test message is received and answered

Apple and Google store steps (V2 only). The native wrapper needs developer accounts, signing keys, store listings, privacy labels that match the help page, and store review, none of which are needed for the MVP or V1.

7. Release procedure by stage

Every release follows the same three phases. Stage-specific steps come after.

Every release

1. **Before.** Confirm the release criteria for the stage are met (section 1). Run the nightly and pre-release checks on staging. Review the migration list, new settings and secrets, and the rollback plan. Take a fresh backup or confirm the latest one is good. Write the short release note.
2. **During.** Merge to main, let staging deploy and pass its smoke test, approve, and watch production through its migrate, deploy and smoke steps. Do not release late in the evening or just before being unavailable, since one person is on call.
3. **After.** Run the production smoke test (open the app, sign in, log a habit, add an expense, record a mood, sync a second device, receive a test reminder). Watch error, sync and scheduler dashboards for the first hours. Announce only after the checks pass.

By stage

Stage	Specific steps	Done when
MVP beta	Use the production environment with an invite-only link, not staging, because testers enter real data. Invite at least 10 testers; tell them what is being tested and how to report problems; check the privacy and disclaimer text first. Review daily logs, errors and sync health in week one	10 testers log 4 or more days a week (the gate); no open severe defects; the beta NFRs hold on real devices
MVP public	Finish the DPDP and GDPR review and the name, domain and trademark check; complete the restore drill; move to paid database and scheduler plans if free-tier limits or terms require it; confirm support contact, helplines and legal pages; lift the invite restriction; keep the previous build ready to redeploy	Launch checks pass; no severe defect in the first week; metrics start to feed the V1 gate
V1	Release in slices: savings goals and the weekly recap first, then step import where the platform allows, then the Plus plan. For billing: set up Razorpay live keys and the webhook address, run test payments in staging, then a small real payment in production and confirm the plan changes and a repeated webhook does nothing; retest export and deletion with the new tables	V1 requirements pass; payments tested end to end; export and deletion tested; day-30 retention near 20%
V2	Build the native wrapper around the same web app; prepare store accounts, signing keys, listings and privacy labels; submit for review and keep the web app live throughout; add languages as message files	Wrapper passes store review; targets hold for two consecutive months

Communicating changes. Users are told in plain words what changed, never with shame or pressure. For anything that touches their data (a migration, a rollback, an incident) the message says what happened, what it means for their data and what, if anything, they need to do.

8. Monitoring, rollback and incident response

One person runs TIKO, so alerts must be few and meaningful, and the system must keep working for users while a problem waits. Thresholds below are proposals to tune with beta data.

What is watched

Signal	Source	Alert when	First action
Availability	Uptime check on a health route (target 99.5% a month, to validate)	Two checks in a row fail	Check the host and database status; roll back if a release just shipped
Errors	Sentry	New error type, or a sharp rise after a release	Compare to the release; roll back if severe
Sync health	Rate of sync responses by status	Rising 5xx, 429 or `stale` results	Check the database and recent migrations
Scheduler	Heartbeat from each job	A job misses its run	Run it manually; reminders catch up because jobs look at what is due
Email and push delivery	Provider dashboards	Bounces, complaints, or many 410 responses	Check the domain records; fall back to the other channel
Database	Provider metrics	Storage, connections or compute near the plan limit	Upgrade or clean up before it fills
Backups	Provider status	A backup fails	Run a manual backup; investigate before the next release

Signal	Source	Alert when	First action
Cost and plan limits	Provider dashboards	Usage near a plan limit	Section 9 triggers
Product health	Analytics	Day-7 and day-30 retention, days logged a week (PRD measures)	Review weekly; informs the gates

Alerts go to the developer by a channel that interrupts (phone notification or email with a rule), production only. Support and data requests have a stated response time once the public launch is set.

Rollback

Situation	Action	Effect on data
A bad build with no schema change	Redeploy the previous build from the host	None
A bad build with an expand-only migration	Redeploy the previous build; leave the added columns in place	None; old code ignores new columns
A migration that broke the database	Stop sync (it answers 503, devices queue entries), fix forward with a new migration; restore from backup only if data was damaged	Devices keep and re-send their entries once sync returns
Data corruption or deletion	Stop writes, restore the latest good backup to a new database, run integrity checks, switch over, then let devices re-send recent entries (see the epoch proposal in section 5)	Entries since the backup return from devices; entries only on the server are lost
Failed scheduler or provider	Switch to the fallback in section 2, or run the job by hand	Reminders may be late, not lost

Because logging happens on the device first, an outage or rollback does not stop people from logging; it only delays their sync. Tell users that when it applies.

Incident response

1. **Detect and contain.** Confirm the problem, stop the harm (roll back, pause sync or a feature through the feature switch, revoke a leaked secret).
2. **Assess.** What data, which users, since when? Check logs and database records. Note times as they are found.
3. **Fix and verify.** Apply the fix through the normal pipeline where possible; run the smoke test and the relevant scenarios from the Testing Plan.
4. **Tell people.** Users and, where a law requires it, the authorities. For a privacy incident (for example journal or mood data exposed, or the master key leaked) this has deadlines: GDPR sets a 72-hour clock for notifying the supervisory authority where it applies, and DPDP has its own duties. Get legal advice before the public launch about who must be told, and what, so this is decided in advance.
5. **Learn.** A short written review within a week: what happened, why it was not caught earlier, one change to prevent it, and a regression test for it.

9. Cost, scaling triggers, risks and open questions

The PRD assumes free-tier infrastructure until usage grows. This section names what to check about free tiers, when to start paying, and what is still undecided. Current plan limits and prices change often and were not checked for this document, so each must be confirmed with the provider before it is relied on.

Check each free tier before relying on it

- **Terms of use.** Some free hosting plans are limited to personal or non-commercial use. A public launch, and certainly a Plus plan, may require a paid plan.
- **Scheduler precision.** Reminders need runs about every 5 minutes; free hosting cron is often much coarser.
- **Idle behavior.** Free databases may pause or slow when idle, adding delay to first requests and risking missed reminders.
- **Storage and bandwidth.** Compare against the capacity estimate in the Database Design: roughly 10 GB of new data a year at 50,000 users, with about 30% active daily (approximate, to validate).

- **Backup retention and region.** Confirm backups can be kept for 30 days and no more, and that the region suits users in India.

When to move to paid plans (triggers)

Trigger	Action
Before the public launch	Production on paid hosting, database and scheduler where the checks above require it
Database storage, connections or compute near the plan limit	Upgrade before it is full, not after
Reminders late or missed because of scheduler limits	Move to a scheduler with 5-minute precision
Active users approaching the planning ceiling of 50,000 on one database (NFR-12)	Plan a redesign; load-test first
Plus plan launches (V1)	Costs now have revenue; revisit plan sizes and the price decision

Risks to the deployment

Risk	Effect	Response
One person deploys and is on call	Slow response; mistakes under pressure	Few alerts; small releases; no late-day releases; written runbooks from this plan
Master encryption key lost or leaked	Journals unreadable, or exposed	Two-place backup and a tested recovery; rotation by key version; incident steps in section 8
A migration damages data	Data loss	Expand then contract; staging rehearsal; restore drill; backups
Free-tier limits or terms block launch	Delay or outage	Check early; budget a paid plan
Backups outlive the deletion promise	Breaks the 30-day promise	Retention at 30 days or less; verify the setting

Risk	Effect	Response
Provider outage	Sync or reminders stop	Local-first app keeps working; fallbacks in section 2
Name or domain unavailable	Rename late	Run the check before the public launch

Open questions that change the deployment

1. Neon with Auth.js, or Supabase for the database and sign-in? The runbooks differ in detail only.
2. Which scheduler, and does its precision meet the 5-minute reminder need on the chosen plan?
3. Do the free hosting and database plans permit a public, eventually paid, product?
4. Should beta run on production with invites (assumed here), and should the Architecture Document be changed to say so?
5. Should the sync API add a database epoch so devices can repair a restored server (proposed in section 5)?
6. Should a small feature-switch table be added so risky features can be turned off without a deploy?
7. What response time does support promise at launch, and who answers when the developer is away?
8. Is a public status page wanted at launch, or only direct messages to users?
9. Where may personal data be stored under DPDP and GDPR, which decides the regions in section 2?